

Лабораторная №12. Протокол MQTT

Цель работы

Изучить протокол MQTT — лёгкий протокол обмена сообщениями для IoT-систем. Освоить архитектуру «издатель-подписчик» (Pub/Sub), научиться настраивать MQTT-брокер Mosquitto на одноплатном компьютере и создавать клиентские приложения на Python и C для публикации и приёма данных.

Оборудование и исходные данные

- Lichee RV Dock с ALT Linux и сетевым подключением;
- компьютер с ALT Linux в той же учебной сети;
- Mosquitto и клиентские утилиты;
- Python 3 с библиотекой Paho MQTT;
- компилятор GCC и libmosquitto-devel для дополнительной части.

Ключевые понятия

Протокол MQTT

Определение

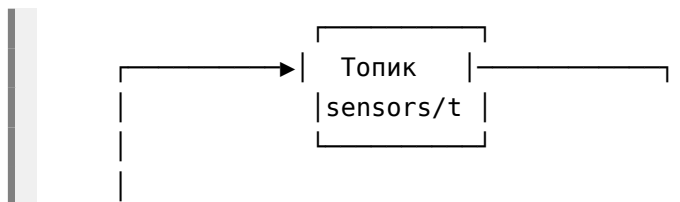
MQTT (Message Queuing Telemetry Transport) — лёгкий протокол обмена сообщениями по модели «издатель-подписчик», применяемый в интернете вещей, автоматизации и телеметрии.

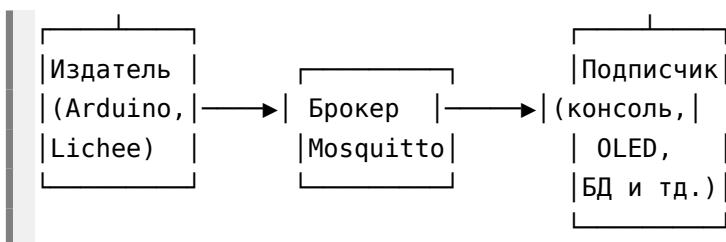
Перечислим основные особенности протокола:

- **Лёгкий протокол** — минимальный заголовок пакета (2 байта), подходит для устройств с ограниченной памятью и медленными каналами
- **Асинхронная модель** — издатель и подписчик не знают друг о друге, взаимодействуют только через брокер
- **Устойчивость к обрывам** — keep-alive, сохранённые сессии, сообщения о разрыве.

Архитектура «издатель-подписчик»

В отличие от клиент-серверной архитектуры, где отправитель напрямую обращается к получателю по адресу, MQTT использует модель Pub/Sub:





Определение

Брокер (broker) — сервер, принимающий сообщения от издателей и доставляющий их подписчикам. В этой работе брокером служит Lichee RV Dock с Mosquitto.

Определение

Издатель (publisher) — клиент, отправляющий сообщения в топик брокера.

Определение

Подписчик (subscriber) — клиент, получающий сообщения из одного или нескольких топиков.

Топики

Определение

Топик (topic) — строковый адрес в иерархии MQTT, уровни которого разделены символом /.

Топик	Данные
home/livingroom/temperature	Температура в гостиной
home/kitchen/humidity	Влажность на кухне
lichee/stats/cpu	Загрузка ЦП одноплатника
sensors/bme280	Все данные с датчика BME280

Шаблонные подписки позволяют подписываться на группы топиков:

- **+** — заменяет ровно один уровень иерархии. Подписка home/+/temperature получит данные о температуре из всех комнат
- **#** — заменяет любое количество уровней (только в конце). Подписка home/# получит все сообщения из всех комнат и подтопиков дома

Качество обслуживания (QoS)

MQTT предоставляет три уровня гарантии доставки сообщений:

Уровень	Название	Описание	Пример использования
QoS 0	At most once	Сообщение отправляется один раз без подтверждения. Возможна потеря.	Телеметрия: температура раз в минуту
QoS 1	At least once	Брокер подтверждает получение (PUBACK). Возможны дубликаты.	Команды управления освещением
QoS 2	Exactly once	Четырёхэтапная транзакция, списание рукопожатие. Гарантирует ровно одну доставку.	

В рамках данной лабораторной работы используется QoS 0 — самый простой и быстрый режим.

Retained-сообщения

Определение

Retained-сообщение — последнее сохранённое брокером сообщение топика, которое автоматически получают новые подписчики.

При публикации с флагом `retain = true` брокер сохраняет последнее сообщение в топике и автоматически отправляет его каждому новому подписчику. Это удобно для статусов, которые меняются редко: новый клиент немедленно получает актуальное значение, не дожидаясь следующей публикации.

Will-сообщения

Определение

Will-сообщение — сообщение, которое брокер публикует при нештатном отключении клиента.

Will-сообщение задаётся при подключении клиента к брокеру. Если клиент отключается нештатно из-за обрыва связи или тайм-аута `keep-alive`,

брокер автоматически публикует это сообщение. Так система мониторинга узнаёт об аварийном отключении устройства.

MQTT-брокер Mosquitto

Определение

Mosquitto — открытая реализация MQTT-брокера, доступная в репозиториях ALT Linux.

Установка и запуск:

```
apt-get install mosquitto
systemctl start mosquitto
systemctl enable mosquitto
```

Проверка работоспособности командной строкой:

```
# В первом терминале запустите подписчика:
mosquitto_sub -h localhost -t "test/topic"

# Во втором терминале запустите издателя:
mosquitto_pub -h localhost -t "test/topic" -m "Hello MQTT!"
```

Проверка

Ожидаемый результат. В терминале подписчика появляется Hello MQTT!.

Python-клиент paho-mqtt

Определение

Eclipse Paho — набор клиентских библиотек MQTT для разных языков программирования.

Установите библиотеку для Python:

```
apt-get install python3-module-paho
```

Основные функции обратного вызова:

```
import paho.mqtt.client as mqtt

def on_connect(client, userdata, flags, rc):
    """Вызывается при подключении к брокеру. rc == 0 — успех."""
    print(f"Connected: {rc}")

def on_message(client, userdata, msg):
    """Вызывается при получении сообщения из подписанного топика."""
    print(f"{msg.topic}: {msg.payload.decode()}")

client = mqtt.Client()
```

```
client.on_connect = on_connect
client.on_message = on_message

client.connect("localhost", 1883)    # адрес брокера, порт
client.subscribe("topic/name")      # подписка на топик
client.loop_forever()               # бесконечный цикл обработки
```

Для публикации используется метод `publish()`:

```
client.publish("topic/name", "data_string")
```

С-клиент libmosquitto

Определение

libmosquitto — клиентская библиотека MQTT для языка C.

```
apt-get install libmosquitto libmosquitto-devel
```

Порядок выполнения

Установка и настройка брокера Mosquitto

Брокер будет запущен на Lichee RV Dock. На ПК установите только клиентские утилиты для тестирования.

1. **На Lichee:** установите Mosquitto и клиентские утилиты:

```
apt-get install mosquitto
```

2. **На ПК (ALT Linux):** установите Mosquitto — пакет включает клиентские утилиты `mosquitto_pub/mosquitto_sub` для тестирования:

```
apt-get install mosquitto
```

3. **На Lichee:** запустите брокер и добавьте в автозагрузку:

```
systemctl start mosquitto
systemctl enable mosquitto
```

4. **На Lichee:** настройте брокер для приёма внешних подключений. По умолчанию Mosquitto слушает только `localhost`. Добавьте в файл `/etc/mosquitto/mosquitto.conf` строки:

Внимание

Настройки `listener 1883 0.0.0.0` и `allow_anonymous true` открывают брокер на всех сетевых интерфейсах без аутентификации. Используйте их только в изолированной учебной сети; не подключайте такой стенд к публичной или недоверенной сети.

```
listener 1883 0.0.0.0
allow_anonymous true
```

В реальной сети необходимо включить аутентификацию, ограничить доступ firewall-правилами и по возможности использовать TLS.

Перезапустите брокер:

```
systemctl restart mosquitto
```

Проверьте, что порт 1883 слушается на всех интерфейсах:

```
ss -tlnp | grep 1883
```

Проверка

Ожидаемый результат. Вывод содержит 0.0.0.0:1883 или *:1883.

5. **Узнайте IP-адрес Lichee** (понадобится для подключения подписчика с ПК):

```
ip address | grep "inet " | grep -v 127.0.0.1
```

или:

```
hostname -i
```

Запишите этот IP-адрес — далее будем обозначать его <IP_LICHEE>.

Проверка

Проверка адреса. Команда `hostname -i` может вывести несколько адресов. Выберите адрес интерфейса в одной учебной сети с ПК и подтвердите его командами `ip address` и `ping <IP_LICHEE>`.

6. **Проверка связности с ПК:**

```
# С ПК проверьте доступность Lichee:
ping <IP_LICHEE>
```

```
# Проверьте, что порт 1883 на Lichee открыт:
telnet <IP_LICHEE> 1883
```

Если соединение устанавливается (в telnet появится Connected или escape-символ) — брокер доступен.

7. **Протестируйте Pub/Sub между Lichee и ПК:**

Терминал 1 — на ПК, подписчик:

```
mosquitto_sub -h <IP_LICHEE> -t "test/hello"
```

Терминал 2 — на Lichee (по SSH), издатель:

```
mosquitto_pub -h localhost -t "test/hello" -m "Hello from Lichee!"
```

Проверка

Ожидаемый результат. В терминале на ПК появляется Hello from Lichee!: брокер принимает внешнее подключение и доставляет сообщение подписчику.

Python-издатель системных метрик

Создайте скрипт `mqtt_publisher.py`, который читает загрузку CPU и использование RAM и публикует их в топик `lichee/stats`:

```
#!/usr/bin/env python3

import paho.mqtt.client as mqtt
import json
import time

TOPIC = "lichee/stats"
INTERVAL = 2.0

def get_cpu_usage():
    """Возвращает загрузку CPU в процентах (от 0 до 100)."""
    with open("/proc/stat") as f:
        fields = f.readline().split()
        user, nice, system, idle = map(int, fields[1:5])
        total = user + nice + system + idle

    # Расчёт по дельте между вызовами
    if not hasattr(get_cpu_usage, "prev"):
        get_cpu_usage.prev = (total, idle)
        return 0.0

    prev_total, prev_idle = get_cpu_usage.prev
    get_cpu_usage.prev = (total, idle)

    diff_total = total - prev_total
    diff_idle = idle - prev_idle
    return 100.0 * (1.0 - diff_idle / diff_total) if diff_total > 0 else 0.0

def get_ram_usage():
    """Возвращает использование RAM в процентах (от 0 до 100)."""
    total = free = 0
    with open("/proc/meminfo") as f:
        for line in f:
            if line.startswith("MemTotal:"):
                total = int(line.split()[1])
            elif line.startswith("MemAvailable:"):
                free = int(line.split()[1])
    return 100.0 * (total - free) / total if total > 0 else 0.0
```

```
def main():
    client = mqtt.Client()
    client.connect("localhost", 1883)
    client.loop_start()

    print(f"Publishing to '{TOPIC}' every {INTERVAL}s. Ctrl+C to stop.")
    try:
        while True:
            cpu = get_cpu_usage()
            ram = get_ram_usage()
            payload = json.dumps({"cpu": round(cpu, 1), "ram": round(ram, 1)})
            client.publish(TOPIC, payload)
            print(f"Published: {payload}")
            time.sleep(INTERVAL)
    except KeyboardInterrupt:
        print("Stopped.")
    finally:
        client.loop_stop()
        client.disconnect()

if __name__ == "__main__":
    main()
```

Объяснение: - `get_cpu_usage()` читает `/proc/stat`, вычисляя загрузку как 100% - доля простоя между двумя вызовами. Первый вызов возвращает 0 (базовый замер), последующие — реальное значение - `get_ram_usage()` читает `/proc/meminfo`, находит поля `MemTotal` и `MemAvailable`, вычисляет процент занятой памяти - Данные упаковываются в JSON и публикуются каждые 2 секунды - `loop_start()` запускает сетевой цикл `runo` в фоновом потоке, не блокируя основной

Запустите скрипт:

```
python3 mqtt_publisher.py
```

Оставьте работать — на следующем шаге мы подпишемся на этот топик.

Python-подписчик на ПК

Скрипт подписчика будет запущен на вашем ПК. Он подключается к брокеру на Lichee по IP-адресу и получает данные из топика `lichee/stats`.

Внимание

Перед запуском подписчика замените `<IP_LICHEE>` фактическим адресом Lichee и проверьте с ПК доступность порта 1883, например командой `telnet <IP_LICHEE> 1883`. Не запускайте скрипт с шаблонным адресом.

Создайте на ПК скрипт `mqtt_subscriber.py`:

```
#!/usr/bin/env python3

import paho.mqtt.client as mqtt
import json

# IP-адрес Lichee RV Dock из шага 1
BROKER = "<IP_LICHEE>"
TOPIC = "lichee/stats"

def on_connect(client, userdata, flags, rc):
    print(f"Connected to broker (rc={rc})")
    client.subscribe(TOPIC)
    print(f"Subscribed to '{TOPIC}'")

def on_message(client, userdata, msg):
    try:
        data = json.loads(msg.payload.decode())
        cpu = data.get("cpu", "?")
        ram = data.get("ram", "?")
        print(f"CPU: {cpu:5.1f}%   RAM: {ram:5.1f}%")
    except json.JSONDecodeError:
        print(f"Raw: {msg.payload.decode()}")

def main():
    client = mqtt.Client()
    client.on_connect = on_connect
    client.on_message = on_message
    client.connect(BROKER, 1883)
    print(f"Connected to MQTT broker at {BROKER}")
    print("Listening for messages...")
    client.loop_forever()

if __name__ == "__main__":
    main()
```

Объяснение: - BROKER — IP-адрес Lichee RV Dock в вашей локальной сети. Замените <IP_LICHEE> на реальный адрес, определённый на шаге 1 - on_connect вызывается при успешном подключении — здесь выполняется подписка на топик - on_message вызывается при получении сообщения: парсинг JSON, извлечение cpu и ram, форматированный вывод - loop_forever() блокирует поток и обрабатывает входящие MQTT-пакеты бесконечно

Установка paho-mqtt на ПК (если не установлена):

```
apt-get install python3-module-paho
```

Запуск:

Проверка

Проверка запуска. Сначала запустите брокер Mosquitto, затем издатель на Lichee и после него подписчик на ПК. Перед следующим этапом убедитесь, что предыдущий работает без ошибок.

1. Убедитесь, что издатель `mqtt_publisher.py` запущен на Lichee (шаг 2)
2. Запустите подписчик на ПК:

```
python3 mqtt_subscriber.py
```

В консоли должно появиться:

```
Connected to broker (rc=0)
Subscribed to 'lichee/stats'
CPU: 12.3% RAM: 45.7%
CPU: 15.1% RAM: 45.8%
...
```

С-клиент-издатель

MQTT-клиент можно написать не только на Python, но и на C с использованием библиотеки `libmosquitto`. Это даёт меньший размер бинарного файла и лучшую производительность.

Проверка

Дополнительная проверка. Сначала проверьте цепочку с Python-издателем, затем замените только издатель C-клиентом и сравните формат сообщений и устойчивость подключения.

Установка библиотек на одноплатнике

```
apt-get install libmosquitto libmosquitto-devel
```

Исходный код Создайте файл `mqtt_c_publisher.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <mosquitto.h>

#define MQTT_HOST "localhost"
#define MQTT_PORT 1883
#define MQTT_TOPIC "lichee/stats"
#define KEEPALIVE 60
```

```

static float get_cpu_usage(void) {
    FILE *fp = fopen("/proc/stat", "r");
    if (!fp) return -1;

    unsigned long user, nice, system, idle;
    fscanf(fp, "cpu %lu %lu %lu %lu", &user, &nice, &system, &idle);
    fclose(fp);

    unsigned long total = user + nice + system + idle;
    static unsigned long prev_total = 0, prev_idle = 0;
    float usage = 0.0;

    if (prev_total > 0 && total > prev_total) {
        float diff_idle = idle - prev_idle;
        float diff_total = total - prev_total;
        usage = 100.0 * (1.0 - diff_idle / diff_total);
    }
    prev_total = total;
    prev_idle = idle;
    return usage;
}

static float get_ram_usage(void) {
    FILE *fp = fopen("/proc/meminfo", "r");
    if (!fp) return -1;

    unsigned long total = 0, available = 0;
    char line[128];
    while (fgets(line, sizeof(line), fp)) {
        if (strstr(line, "MemTotal:"))
            sscanf(line, "MemTotal: %lu kB", &total);
        if (strstr(line, "MemAvailable:"))
            sscanf(line, "MemAvailable: %lu kB", &available);
    }
    fclose(fp);
    return (total > 0) ? 100.0 * (total - available) / total : -1;
}

int main(void) {
    mosquitto_lib_init();

    struct mosquitto *mosq = mosquitto_new(NULL, true, NULL);
    if (!mosq) {
        fprintf(stderr, "Error: mosquitto_new failed\n");
        return 1;
    }

    if (mosquitto_connect(mosq, MQTT_HOST, MQTT_PORT, KEEPALIVE)) {
        fprintf(stderr, "Error: cannot connect to broker\n");
    }
}

```

```

        mosquitto_destroy(mosq);
        mosquitto_lib_cleanup();
        return 1;
    }

    printf("C MQTT publisher started.\n");
    printf("Publishing to '%s'. Press Ctrl+C to stop.\n", MQTT_TOPIC);

    char payload[128];
    while (1) {
        float cpu = get_cpu_usage();
        float ram = get_ram_usage();

        if (cpu < 0 || ram < 0) {
            fprintf(stderr, "Error reading system stats\n");
            sleep(1);
            continue;
        }

        snprintf(payload, sizeof(payload),
                 "{\"cpu\":%.1f,\"ram\":%.1f}", cpu, ram);

        int ret = mosquitto_publish(mosq, NULL, MQTT_TOPIC,
                                    strlen(payload), payload, 0, false);
        if (ret != MOSQ_ERR_SUCCESS)
            fprintf(stderr, "Publish error: %s\n", mosquitto_strerror(ret));
        else {
            printf("Sent: %s\n", payload);
            ret = mosquitto_loop(mosq, 1000, 1);
            if (ret != MOSQ_ERR_SUCCESS) {
                fprintf(stderr, "Network error: %s\n", mosquitto_strerror(ret));
                break;
            }
        }

        sleep(2);
    }

    mosquitto_destroy(mosq);
    mosquitto_lib_cleanup();
    return 0;
}

```

Нативная компиляция на Lichee

```

gcc -o mqtt_c_publisher mqtt_c_publisher.c -lmosquitto
./mqtt_c_publisher

```

Исходный файл `mqtt_c_publisher.c` копируется на Lichee через `scp`, ком-

пилируется нативно командой выше и запускается прямо на одноплатнике.

Задание

Ознакомьтесь с ключевыми понятиями и выполните следующие подзадачи.

1. **Базовая задача.** Установите и запустите MQTT-брокер Mosquitto на Lichee RV Dock. Настройте брокер на приём подключений со всех сетевых интерфейсов (listener 1883 0.0.0.0). Протестируйте публикацию и подписку между ПК и Lichee через утилиты mosquitto_pub и mosquitto_sub.
2. **Основная задача.** Реализуйте распределённую систему мониторинга загрузки одноплатника:
 - На Lichee: напишите Python-скрипт-издатель, публикующий загрузку CPU и использование RAM (в процентах) в топик lichee/stats раз в 2 секунды. Данные передавайте в формате JSON: {"cpu": 12.3, "ram": 45.7}.
 - На ПК: напишите Python-скрипт-подписчик, подключающийся к брокеру на Lichee (по IP-адресу), принимающий данные из топика lichee/stats и выводящий их в консоль в читаемом виде: CPU: 12.3% RAM: 45.7%.
 - Продемонстрируйте одновременную работу издателя (на Lichee) и подписчика (на ПК).
3. **Дополнительная часть (С-клиент).** Напишите аналог MQTT-издателя на языке C, публикующий те же системные метрики. Выполните нативную компиляцию программы и проверьте её работу на Lichee.
4. **Ответить на контрольные вопросы (письменно, в отчёте).**
5. **Продемонстрировать работу преподавателю.**

Контрольные вопросы

1. В чём преимущества архитектуры «издатель-подписчик» по сравнению с клиент-серверной для IoT-систем?
2. Какие уровни QoS существуют в MQTT и в каких сценариях каждый из них целесообразно использовать?
3. Как обеспечивается безопасность в MQTT? Какие механизмы аутентификации и шифрования поддерживаются?

4. Чем отличается MQTT от других протоколов IoT, таких как CoAP или HTTP/2?
5. Как организовать иерархию топиков для системы умного дома?
6. Какие проблемы могут возникнуть при использовании MQTT в сетях с нестабильным соединением и как их решить?
7. Как масштабировать MQTT-инфраструктуру при увеличении количества устройств?
8. В чём преимущества использования MQTT поверх WebSocket для веб-приложений?
9. Как реализовать retained- и will-сообщения в MQTT и для чего они используются?
10. Какие библиотеки для работы с MQTT существуют для различных платформ?

Требования к отчёту

Отчёт должен содержать:

- цель работы;
- схему взаимодействия издателя, брокера и подписчика;
- команды настройки и проверки Mosquitto;
- исходные тексты клиентов;
- примеры опубликованных и полученных сообщений;
- ответы на контрольные вопросы;
- краткий вывод.

Отправьте отчёт в согласованном формате до установленного срока.

Полезные ссылки

- Официальный сайт MQTT
- Документация Mosquitto
- Eclipse Paho Python
- Документация libmosquitto